

ROS2

DAY 4 – Manual 1

ROS2 Services – Fundamentals & Setup

Objective

By the end of this manual, you will be able to:

1. Understand the concept of ROS2 services.
 2. Know the difference between Topics and Services.
 3. Create a ROS2 CMake interface package for .srv files.
 4. Create a Python package with ROS2 nodes using the service.
 5. Update package.xml and CMakeLists.txt correctly for interface and Python packages.
 6. Build the workspace and run server/client nodes.
-

Step 0 – Pre-Requisites

Before starting:

- ROS2 installed (Foxy, Humble, Rolling)
 - Python 3 installed
 - A ROS2 workspace (ros2_ws) created
 - Basic understanding of ROS2 nodes and topics (from Day 1 & 2)
-

Step 1 – Understand ROS2 Services

1.1 What is a Service?

- A service allows a client node to send a request to a server node and wait for a response.
- Unlike topics (continuous streaming), services are synchronous.

Example:

CLIENT → "Hello server"

SERVER → "hello client... Got your req"

1.2 Service vs Topic

Feature	Topic	Service
Communication	One-way stream	Two-way request/response
Frequency	Continuous	Only when requested
Example	Sensor data	Commands, calculations

1.3 Anatomy of a .srv File

- .srv files define request and response structure.

Structure:

```
# Request
<string> input
---
# Response
<string> output
```

- Service file: TestService.srv

```
string input
---
string output
```

Step 2 – Create the Interface Package (service_pkg)

1. Open a terminal.
2. Navigate to workspace source folder:

```
cd ~/ros2_ws/src
```

3. Create CMake-based package:

```
ros2 pkg create --build-type ament_cmake service_pkg
```

4. Folder structure:

```
service_pkg/  
  CMakeLists.txt  
  package.xml  
  srv/  
  src/
```

Step 2.1 – Create .srv File

```
cd service_pkg  
mkdir srv  
nano srv/TestService.srv
```

Add:

```
string input  
---  
string output
```

Top = request (input), bottom = response (output).

Step 2.2 – Update CMakeLists.txt

1. Add dependencies:

```
find_package(ament_cmake REQUIRED)  
find_package(rosidl_default_generators REQUIRED)
```

2. Generate service interfaces:

```
rosidl_generate_interfaces(${PROJECT_NAME}  
  "srv/TestService.srv"  
  DEPENDENCIES builtin_interfaces  
)
```

3. Export dependencies:

```
ament_export_dependencies(rosidl_default_runtime)
```

4. Finalize:

```
ament_package()
```

Step 2.3 – Update package.xml

Add the following inside <package>:

```
<buildtool_depend>rosidl_default_generators</buildtool_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

This marks the package as a ROS interface package and ensures downstream packages (Python nodes) can use the generated service.

Step 3 – Create the Python Node Package (my_py_pkg)

1. Navigate to workspace source folder:

```
cd ~/ros2_ws/src
```

2. Create Python package:

```
ros2 pkg create --build-type ament_python my_py_pkg
```

3. Folder structure:

```
my_py_pkg/
  setup.py
  package.xml
  my_py_pkg/
    __init__.py
    srv_server.py
    srv_client.py
  test/
```

Step 3.1 – Update package.xml Dependencies

Add:

```
<build_depend>service_pkg</build_depend>
<exec_depend>service_pkg</exec_depend>
```

- Ensures my_py_pkg can import TestService from service_pkg.

Step 3.2 – Update setup.py

Include .srv file and entry points:

```
data_files=[
    ('share/my_py_pkg/srv', ['srv/TestService.srv']),
],
entry_points={
    'console_scripts': [
        'srv_server = my_py_pkg.srv_server:main',
        'srv_client = my_py_pkg.srv_client:main',
    ],
},
```

Step 4 – Add Server Node (srv_server.py)

```
import rclpy
from rclpy.node import Node
from service_pkg.srv import TestService

class ServiceServer(Node):
    def __init__(self):
        super().__init__('ServiceServerNode')
        self.srv = self.create_service(TestService, 'test_service', self.callback)
        self.get_logger().info("Service Server Ready: test_service")

    def callback(self, request, response):
        self.get_logger().info(f"Incoming request: {request.input}")
        response.output = "hello client... Got your req"
        self.get_logger().info(f"Sending response: {response.output}")
        return response

def main(args=None):
    rclpy.init(args=args)
    node = ServiceServer()
```

```
rclpy.spin(node)
rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Step 5 – Add Client Node (srv_client.py)

```
import rclpy
from rclpy.node import Node
from service_pkg.srv import TestService

class ServiceClient(Node):
    def __init__(self):
        super().__init__('ServiceClientNode')
        self.client = self.create_client(TestService, 'test_service')
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().warn("Service not available... waiting")

    def send_request(self, req):
        request = TestService.Request()
        request.input = req
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future)
        return future.result()

def main(args=None):
    rclpy.init(args=args)
    client = ServiceClient()
    response = client.send_request("Hello server")
    print(f"Result from server: {response.output}")
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Step 6 – Build the Workspace

1. Build interface package first:

```
cd ~/ros2_ws
colcon build --packages-select service_pkg
source install/setup.bash
```

2. Then build Python package:

```
colcon build --packages-select my_py_pkg  
source install/setup.bash
```

#This ensures my_py_pkg can import TestService.

Step 7 – Run Server and Client

1. Terminal 1 – server:

```
ros2 run my_py_pkg srv_server
```

2. Terminal 2 – client:

```
ros2 run my_py_pkg srv_client
```

Expected Output:

```
Service Server Ready: test_service  
Incoming request: Hello server  
Sending response: hello client... Got your req  
Result from server: hello client... Got your req
```

Step 8 – Concepts Recap

- service_pkg → CMake interface package, holds .srv file and generates interfaces.
 - my_py_pkg → Python package, contains server and client nodes.
 - Build order: interface first, then Python nodes.
 - .srv defines request/response data structure.
 - Dependencies in package.xml and setup.py ensure everything links correctly.
 - Server waits for requests, client sends requests, communication is synchronous.
-

✓ Manual 1 Complete!

Next: Manual 2 will cover line-by-line code explanation, testing, and advanced tips for ROS2 services.
